Fio bot navigation: a level designer’s guide

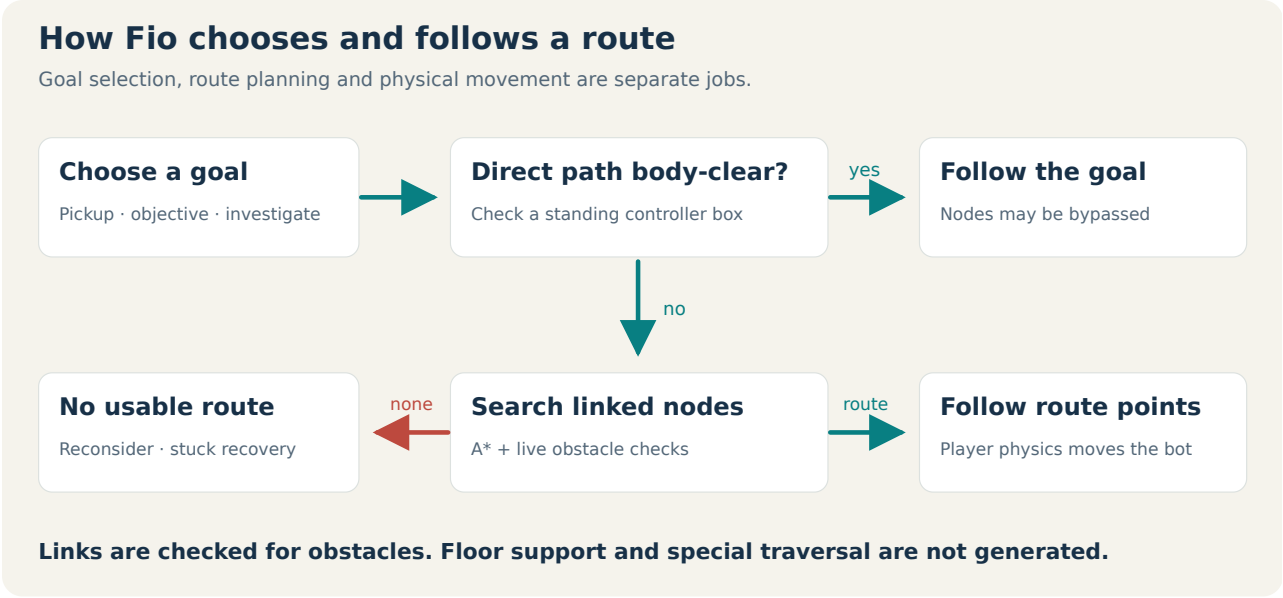
How bots choose a path, where to place Navigation Nodes, and how to build routes that work in Deathmatch and Capture the Flag.

Applies to the current `Deathmatch` branch, inspected at `3ae34c6`. This guide describes the existing authored waypoint system. Automatic navigation with optional authored hints is planned; it is not implemented in this version. Quake III’s AAS navigation and fuzzy bot decisions are references for that work, not features already available in Fio.

Download the printable PDF.

1. What a bot actually follows

A **node** marks a useful position on the floor. A **link** tells the planner that two nodes are intended to connect. A **route** is the ordered list of positions the bot will follow to its goal.



The current sequence is:

1. **Choose a goal.** Depending on the situation, a bot seeks a pickup, follows a CTF objective, investigates an opponent’s last known position, or roams between nodes. Visible enemies can trigger direct combat movement instead of waypoint following.
2. **Try the direct route.** If the controller’s full body can clear the straight segment, the planner returns the goal directly. Nodes are not mandatory stops: a clear direct path takes precedence over them.
3. **Attach to the graph.** When the direct route is blocked, the planner considers up to three nearest body-clear start nodes. Goal-side nodes also need a clear connection to the goal. The bot and goal do not need to stand exactly on nodes.

4. **Run A*.** The planner adds the distance already travelled to an estimate of the remaining distance. Both use straight-line 3D distance; the estimate helps it search toward the goal. Every authored link is checked for current obstacle clearance before use.
5. **Move and reconsider.** The bot steers toward successive positions using Fio's Player controller. Routes are reconsidered periodically, and stuck recovery attempts a jump and turn. Recovery does not replace a connected, walkable route.

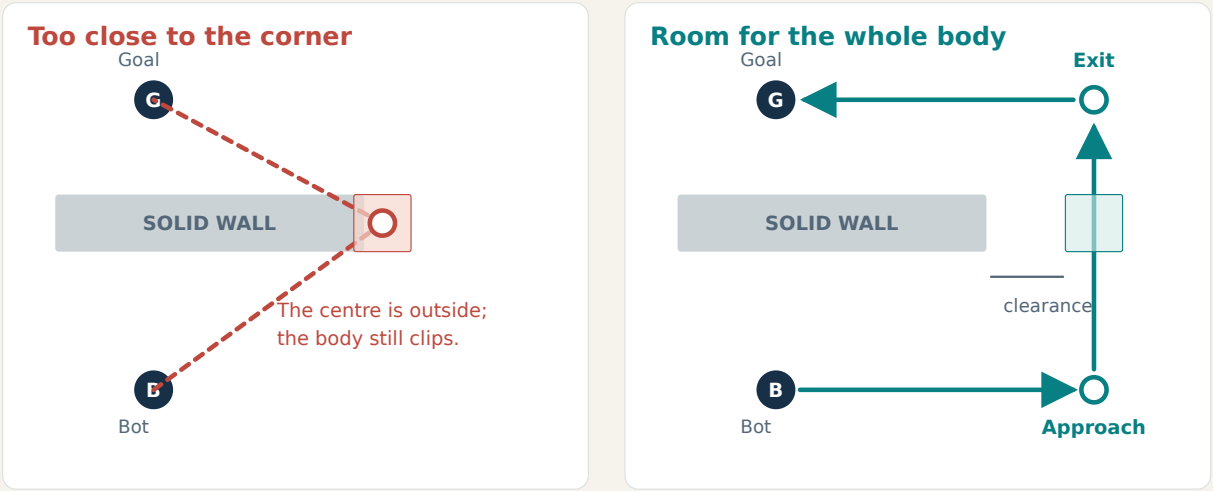
A link expresses intent, not a guarantee. The planner checks an expanded body-sized box against brush obstacles. It does not verify continuous floor support or generate special jump, ladder, lift or teleport traversal instructions.

2. Put nodes at changes in movement

Start with **corners, junctions, doorway approaches, entrances to important rooms, and objective approaches**. Add a node where the bot needs a new clear segment, rather than filling every open room with a dense grid.

Corners: clear centres are not enough

Leave a body-sized corridor around every wall edge.



Top-down view. Check the complete link, not just the two node positions.

For each node and every link approaching it:

- Place the node at **floor/feet height**, not at eye height or the centre of the visible sprite.
- Keep the bot's whole body away from walls. The default standing controller is **50 units wide, 100 high and 50 deep**. Its centre needs more than 25 units of lateral clearance; exact contact leaves no working margin. Around ordinary corners, **40–64 units of clearance** is a useful starting point, then test the complete segment.
- Put nodes far enough beyond corners that the joining segment clears the wall with the whole body. Two valid endpoint positions can still have an invalid diagonal between them.
- Place a node on **each side of a doorway**, outside the door's travel volume. Check headroom and width. A closed door can invalidate a link; bots do not gain a door-activation plan merely because a node is nearby.

- Add intermediate nodes where a corridor bends. In a large open room, a few well-connected entrance/junction nodes are usually more useful than many collinear nodes.

There is **no required node spacing**. As a design starting point, try 128–256 units between important indoor decisions, then adjust for the geometry. Those values are suggestions, not engine thresholds.

Worked example: route around one wall

Coordinates are `(X, Y, Z)`, with Y up. Suppose the floor is at Y = 0, a wall is centred at `(0, 120, 150)` with size `(300, 240, 50)`, the bot starts at `(0, 0, 0)`, and the goal is `(0, 0, 300)`.

Node name	Feet position	`links`
`East_Approach`	`(240, 0, 0)`	`East_Exit`
`East_Exit`	`(240, 0, 300)`	empty

The bot can attach to `East_Approach`, pass east of the wall, then leave `East_Exit` for the goal. The wall ends at X = 150, so X = 240 leaves 90 units to its east edge. This layout matches the geometry exercised by Fio's existing routing test.

3. Create and connect nodes in the editor

For a playable arena, enable the Deathmatch plugin and provide **Match Settings**, a **Bot Opponent**, a normal **PlayerStart**, and suitable **Respawn Points**. Navigation Nodes alone do not activate a match or spawn bots.

1. Place a **Navigation Node** from the **Plugins → deathmatch** entity menu.
2. Move it onto a walkable floor position. Confirm its height in a side view.
3. Give it a **unique, case-sensitive name without spaces**, such as `West_Junction`.
4. In its `links` property, enter neighbouring node names separated by commas or spaces: `North_Door, Health_Approach`.
5. Repeat at the next corner or junction, then test the connecting segment in both directions.

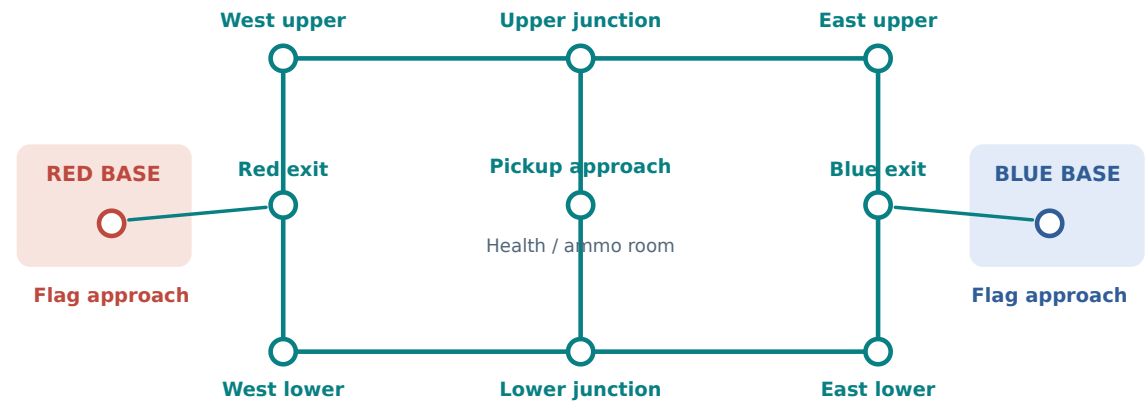
Links are bidirectional. Listing `North_Door` in `West_Junction.links` creates both directions automatically; you do not have to repeat the reverse entry. This also means a link cannot describe a safe one-way drop. Misspelled names and self-links are ignored, while duplicate node names overwrite a graph entry. Use distinctive, consistent names.

4. Build a network, not a single chain

Deathmatch needs access to useful rooms and pickups. CTF needs reliable travel between bases, flag recovery positions and several approaches. Give important areas at least two routes where the map geometry allows it.

Connect objectives with loops and useful branches

CTF connectivity sketch. Floor geometry and obstacle clearance still need to be checked.



Two base-to-base routes + a central connection. Each drawn link must match a real walkable corridor.

The diagram is a **connectivity sketch**: each line still needs body clearance and floor support in your actual level. Match links to actual corridors; never draw a connection through cover simply to make the graph look complete.

Location	Best placement
Respawn area	Connect its safe exit to the main network. Spread spawns out and avoid forcing all traffic through one occupied spawn point.
Pickup	Put an approach node on the floor within usable reach, with a clear final approach. A pickup does not need a node at the centre of its sprite.
CTF flag/base	Connect a floor-level approach near the flag to both the attack routes and local defensive routes. Keep the flag's touch area unobstructed.
Defensive position	Use a position with useful sightlines that remains connected to flag recovery and exit routes. A node itself does not assign a defender role.
Door/chokepoint	Provide a genuinely traversable alternate route if one exists. Check what happens when the door is shut.

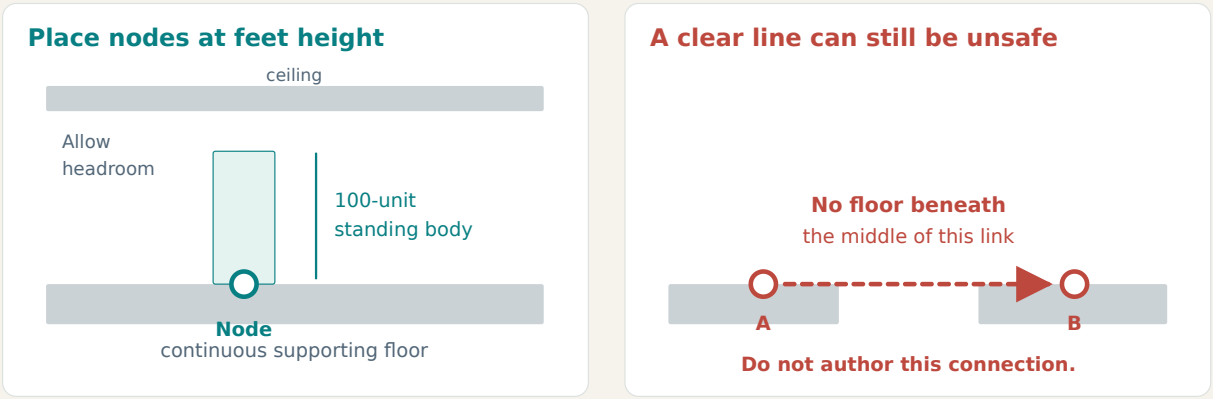
Current pickup limitation: bot item selection filters out occluded pickups before asking for a route. A hidden health pack may remain unselected even if nodes reach it. Keep useful pickups visible from common approaches when testing the current implementation.

CTF requires exactly one red and one blue **CTF Flag**, plus appropriate team spawns and Match Settings. A good node network helps bots reach their objectives; it does not create flag rules, choose team roles, or turn a map into CTF by itself.

5. Check floor support and vertical changes

Height, headroom and gaps

The current planner checks obstacle clearance, not continuous ground support.



Side view. Walk each segment in both directions; a bidirectional link cannot encode a one-way drop.

The most important safety check is **whether a player can actually walk every linked segment in both directions**.

- Use side views to check node height, ceiling clearance and the floor below the entire segment.
- Do not connect across pits or between overlapping floors just because their top-down positions are close. A body-clear line over empty space may be accepted and lead the bot off the edge.
- For ordinary stairs and ramps, put nodes at useful landings and bends, then test short links in both directions. Player physics supports steps and slopes, but the planner's conservative brush-box approximation can still reject a physically walkable segment.
- Do not depend on planned jumps, ladders, lifts or teleports. The current graph has no movement-type links for them. A stuck bot's jump attempt is reactive recovery, not a validated jump route.
- Keep routes out of damaging or otherwise unsafe areas yourself. The current A* cost measures distance, not damage exposure, tactical risk or travel time.

6. Test and diagnose the network

Test in Play with a bot and a useful destination on opposite sides of the geometry. Remove convenient direct sightlines where you want to exercise the waypoint route. For CTF, test both teams, flag carrying, dropped-flag recovery and return trips. Test doors open and closed.

Symptom	Check first
Bot takes a direct path past your nodes	A clear direct route takes precedence; nodes are not forced patrol stops.
Bot stops at a wall or keeps turning/jumping	Check the missing corner node, link spelling, body clearance and whether the graph actually connects around the obstacle.

Symptom	Check first
Bot falls into a gap	Inspect floor support along the link. Obstacle clearance alone does not make a walkable route.
Bot cannot reach another floor	Check a continuous, tested walking route. A vertical link does not add jump, ladder or lift behaviour.
Bot ignores a useful pickup	Check inventory need, pickup availability, visibility and a usable final approach, as well as the route.
Door closure strands a bot	Confirm that an alternate connected corridor exists and is open. Waypoints do not automatically activate doors.

Finish by walking every link yourself, testing both directions, and watching bots use the network over several lives. Remove redundant nodes only after confirming that the remaining links still provide the required clearance and connections.

Reference: where this behaviour lives

- Navigation graph and A*: name-based links, direct route check and reachable start/goal connections.
- Deathmatch runtime: body clearance, goal following, perception, movement and stuck recovery.
- Bot supplies: pickup desirability, visibility filtering and route-distance scoring.
- CTF/team logic: flag state and bot objective selection.
- Routing tests: `test_waypoints_route_around_wall_and_reject_closed_path`.

Quake III separates **reachability/routing** from **goal desirability** and **behaviour states**. Its AAS routing, fuzzy weights, and CTF team decisions are useful references. Fuzzy logic helps choose what to pursue; it does not establish that the floor, doorway or jump is traversable.

Diagrams and guide prepared for Fio. Current authored waypoint implementation: Deathmatch / 3ae34c6. All diagrams are schematic.